Principal Component Analysis for Network Intrusion Detection Using Machine Learning

Zar Chi Phyo
ID number: 40337033
github link: https://github.com/Zarchi1992/INSE-6220-2254-WinterSemester-40337033

***Abstract*—In recent years, network security has become a critical concern due to the increasing number and sophistication of cyber-attacks. Intrusion Detection Systems (IDS) play a vital role in identifying malicious activities within network traffic and protecting sensitive information. However, the high dimensionality and complexity of network data can negatively impact the performance of machine learning-based IDS. This study presents a comprehensive analysis of network intrusion detection using machine learning techniques on the NSL-KDD dataset. A systematic framework is developed, including data preprocessing through standardization, followed by dimensionality reduction using Principal Component Analysis (PCA). The effectiveness of PCA is further analyzed through various visualization techniques such as explained variance, PCA scatter plots, biplots, and box plots to better understand feature representation. To evaluate the impact of dimensionality reduction, multiple classification models are implemented using the PyCaret framework under two scenarios: with and without PCA. The models are trained and tested on a five-class classification problem, including normal, denial-of-service (DoS), probe, remote-to-local (R2L), and user-to-root (U2R) attacks. The performance of the models is compared using standard evaluation metrics such as accuracy, precision, recall, and F1-score. After model selection and tuning, Explainable Artificial Intelligence (XAI) is incorporated using SHAP (Shapley Additive Explanations) to interpret the behavior of the best classifier. Experimental results demonstrate that dimensionality reduction using PCA can improve computational efficiency while maintaining competitive classification performance. The comparative analysis provides insights into the trade-offs between model complexity and detection capability, highlighting the effectiveness of machine learning-based approaches for intrusion detection.**

***Index Terms*—Intrusion Detection System, NSL-KDD, Machine Learning, Principal Component Analysis, Dimensionality Reduction, Classification.**

I. Introduction

With the rapid growth of computer networks and the increasing reliance on internet-based services, securing network infrastructures against cyber threats has become a critical challenge and posing serious threats to individuals, organizations, and governments. Intrusion Detection Systems (IDS) have emerged as an essential security mechanism for monitoring network traffic and identifying malicious activities in real time [1], [2]. Traditional IDS techniques, such as firewalls and signature-based and statistical approaches, suffer from limitations including high false alarm rates, inability to detect unknown attacks, and poor adaptability to dynamic network environments [3].

IDS techniques can be broadly categorized into signature-based and anomaly-based approaches. Signature-based IDS rely on predefined attack patterns and are effective at detecting known threats; however, they fail to identify novel attacks and require frequent database updates. In contrast, anomaly-based IDS establish a baseline of normal network behavior and flag deviations as potential intrusions. While this approach enables the detection of previously unseen attacks, it often suffers from a high false-positive rate due to the complexity and variability of network traffic.

To address these challenges, machine learning (ML) techniques have been widely adopted in intrusion detection due to their capability to learn patterns from data and detect both known and unknown attacks effectively [4]. Supervised learning algorithms, in particular, have demonstrated strong performance in both binary classification (normal vs. attack) and multi-class intrusion detection. Classical ML models such as Random Forest (RF), K-Nearest Neighbors (KNN), and Support Vector Machines (SVM) have been widely adopted due to their robustness, interpretability, and effectiveness on structured tabular datasets.

However, one of the major challenges in applying ML techniques to intrusion detection is the high dimensionality and complexity of network traffic data. High-dimensional feature spaces not only increase computational cost but also degrade classification performance due to redundant and irrelevant features [6]. To overcome this issue, dimensionality reduction techniques such as Principal Component Analysis (PCA) are commonly employed. PCA transforms the original feature space into a lower-dimensional representation while preserving the most significant variance in the data, thereby improving model efficiency and performance [7].

Recent studies have demonstrated the importance of dimensionality reduction and feature engineering for IDS design. [12] proposed a PCA-Logistic Regression framework for intrusion detection, showing that PCA can be effectively combined with a classical supervised classifier to improve IDS performance, although their analysis mainly focused on the LR model. The authors in [13] conducted a comparative study on the NSL-KDD dataset using multiple ML models and hyperparameter optimization through Grid Search CV, highlighting that careful model selection and tuning can significantly improve classification accuracy and reduce false positives. [14] investigated dimensionality reduction for IDS using both PCA and Auto-Encoder-based feature extraction before applying several classifiers, demonstrating that low-

dimensional representations can preserve strong predictive performance while reducing feature complexity. [15] further examined the impact of different PCA variants for intrusion detection on KDD Cup 99 and NSL-KDD, emphasizing that the choice of feature extraction technique directly affects metrics such as detection rate, F-measure, and false positive rate. In addition, [16] analyzed NSL-KDD using the CRISP-DM methodology and several ML models, reporting that Decision Tree achieved the strongest testing performance among the compared approaches. Similarly, [17] employed feature selection with ensemble-based Random Forest and performed intrusion classification using SVM, Logistic Regression, and KNN on NSL-KDD benchmark data, showing that feature reduction and model choice strongly influence validation accuracy. In addition to improving performance, understanding the decision-making process of ML models has become increasingly important. Many powerful models operate as black-box systems, making it difficult to interpret their predictions. Recently, Explainable Artificial Intelligence (XAI) techniques, such as SHAP (Shapley Additive Explanations), have been introduced to provide both global and local interpretability of ML models by quantifying the contribution of each feature to the prediction [8].

In this report, we present a comprehensive framework for network intrusion detection using the NSL-KDD dataset, a benchmark intrusion detection dataset designed to address redundancies and biases present in the original KDD Cup 1999 dataset [5]. The proposed approach includes data preprocessing through feature standardization, followed by dimensionality reduction using PCA. The effectiveness of PCA is analyzed using various visualization techniques, including explained variance plots, PCA scatter plots, biplots, and box plots. Furthermore, multiple machine learning models are implemented and compared using the PyCaret framework under two scenarios: with and without PCA. The classification task is performed on five classes using comprehensive evaluation metrics such as Accuracy, Precision, Recall, F1-score, ROC–AUC, confusion matrices, and learning curves. Finally, Explainable AI techniques based on SHAP values are applied to interpret the behavior of the selected model, providing insights into feature importance and decision-making processes.

The remainder of this report is organized as follows. Section II presents the methodology framework, including the dataset, preprocessing techniques, PCA, evaluation metrics, and classification algorithms. Section III provides the simulation results and discussions, including PCA analysis, classification performance and the explainable AI analysis using SHAP. Finally, Section IV concludes the report and outlines future research directions.

II. Methodology Framework

In this section, the overall framework of the proposed intrusion detection system is presented. The methodology consists of several stages including dataset description, data preprocessing, dimensionality reduction using Principal Component Analysis (PCA), classification using machine learning algorithms, and evaluation of model performance. The complete workflow of the proposed framework is illustrated in Fig.1.

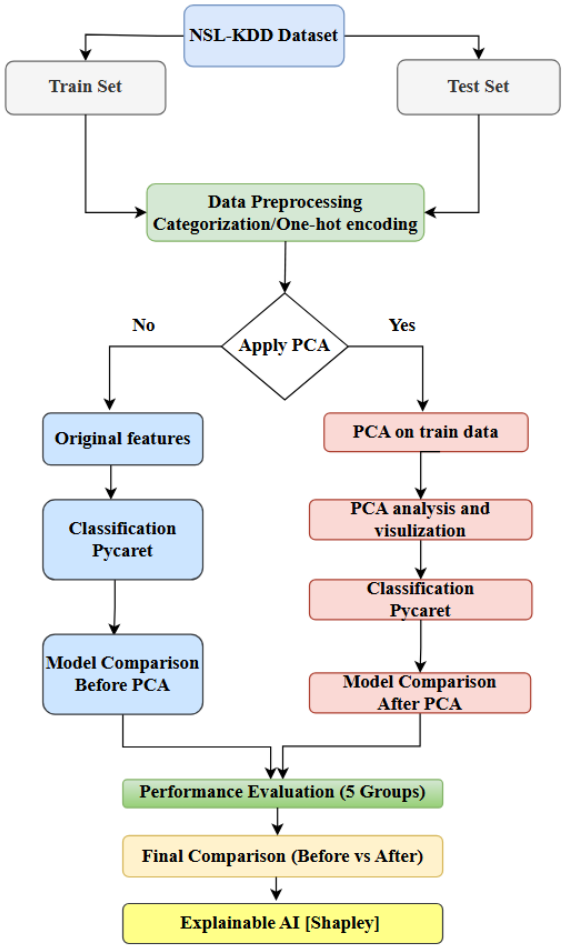


Fig. 1: Methodology Framework

A. A. Dataset

The dataset used in this study is the NSL-KDD dataset, which is publicly available on Kaggle [11]. It is a widely recognized benchmark dataset developed to address the redundancy and evaluation bias present in the original KDD Cup 1999 dataset. The NSL-KDD dataset consists of two subsets: KDDTrain+.txt for training and KDDTest+.txt for testing. Each record represents a network connection and is described by 41 feature attributes along with one target label, resulting in a total of 42 attributes per instance. The model is trained on the training subset and evaluated on the testing subset, ensuring an unbiased and realistic assessment of model performance.

The dataset categorizes network traffic into five classes: normal, denial-of-service (DoS), probe, remote-to-local (R2L), and user-to-root (U2R) attacks [13]. These categories represent different types of malicious activities commonly observed in network environments. DoS attacks involve the exhaustion of

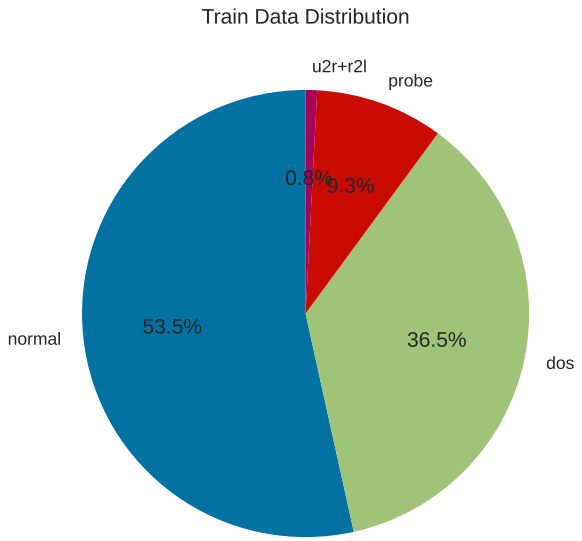


Fig. 2: Train Data Distribution

system or network resources to prevent legitimate users from accessing services. Probe attacks are associated with scanning and surveillance activities, where attackers examine the network to gather information for potential future attacks. R2L attacks occur when an attacker attempts to gain unauthorized access from a remote machine to a local system, often by exploiting vulnerabilities or guessing credentials. U2R attacks involve an attacker obtaining privileged (root-level) access after initially gaining normal user access, allowing full control over the system. The distribution of these classes in both the training and testing datasets is presented in Table I.

In the training dataset, the majority of instances belong to the normal and DoS classes, whereas R2L and U2R are significantly underrepresented, leading to a highly imbalanced class distribution. Similarly, the testing dataset contains both known and previously unseen attack patterns, further increasing the complexity of the classification task.

Fig. 2 illustrates the percentage distribution of normal and attack classes in the training dataset. The imbalance between majority and minority classes highlights the challenges associated with accurately detecting rare attack types such as U2R and R2L, which are often overshadowed by more frequent classes.

TABLE I: NSL-KDD Dataset Distribution

Attack Type	Training Set	Test Set
Normal	67343	9711
DoS	45927	7460
Probe	11656	2421
R2L	995	2885
U2R	52	67

B. B. Data Preprocessing

Data preprocessing is an important step in preparing the dataset for machine learning algorithms. The NSL-KDD dataset contains both numerical and categorical features, which require appropriate transformation before model training.

First, categorical features are converted into numerical representations using encoding techniques. Then, feature scaling is applied using standardization to normalize the data distribution. Standardization transforms the features to have zero mean and unit variance, ensuring that all features contribute equally to the learning process.

C. C. Dataset Description

Fig. 3 illustrates the box plot of the top 15 correlated features after preprocessing. It can be observed that the distributions of the features vary significantly, with some features showing a relatively symmetric spread around the median, while others exhibit skewness. Although a few features appear to follow an approximately normal distribution, most of them deviate from perfect normality, indicating the presence of skewed data. Additionally, outliers are clearly present in several features, particularly in attributes such as *srv_rerror_rate*, *dst_host_count*, and *service_http*. These outliers are distributed on both the upper and lower ends, suggesting extreme values in the dataset, which may influence model performance.

Fig. 4 presents the box plot of selected features grouped by attack categories (normal, DoS, probe, R2L, and U2R). From this figure, it is evident that the feature distributions differ significantly across attack groups, indicating that these features are useful for distinguishing between different types of network traffic. However, similar to Fig. 3 , the distributions are not strictly normal, as many features show skewness and varying spreads across groups. Outliers are also present in all groups, particularly in minority classes such as R2L and U2R, where extreme values are more prominent. This is likely due to the imbalanced nature of the dataset and the rarity of certain attack types.

Overall, the presence of outliers in both figures highlights the variability and complexity of network intrusion data. These outliers may represent rare but important attack patterns, and therefore should be carefully considered rather than simply removed during preprocessing.

Fig. 5 shows the correlation matrix of the top 10 most correlated features selected for visualization in this report. For clarity and readability, only the most significant features are presented, while the full correlation matrix of all 41 features and the group-wise correlation matrices for each attack category are provided in the Google Colab notebook. It can be observed that several feature pairs exhibit strong positive and negative correlations, particularly among attributes such as *serror_rate*, *srv_serror_rate*, and *dst_host_serror_rate*, indicating redundancy in the dataset. Such high correlations suggest that multiple features carry overlapping information, which can negatively affect model performance. Therefore, the use of PCA is justified, as it transforms these correlated features

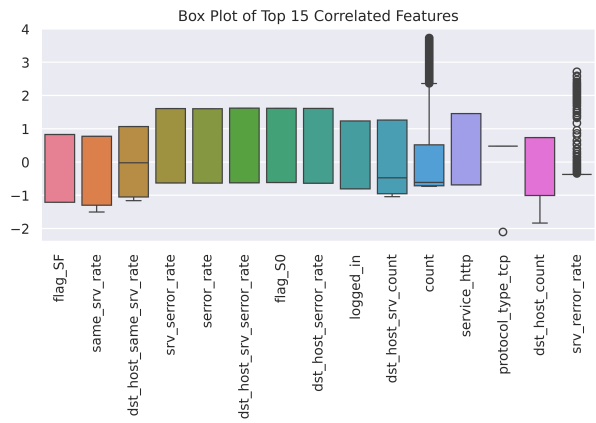


Fig. 3: Box Plot

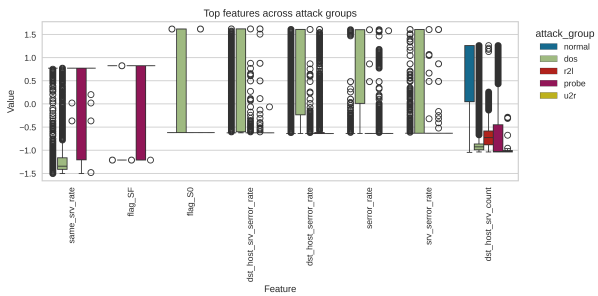


Fig. 4: Box Plot by Groups

into a smaller set of uncorrelated principal components while preserving the most important variance in the data.

D. D. Principal Component Analysis

Principal Component Analysis (PCA) is a widely used dimensionality reduction technique that transforms high-dimensional data into a lower-dimensional space while preserving the most significant variance [7]. PCA is particularly

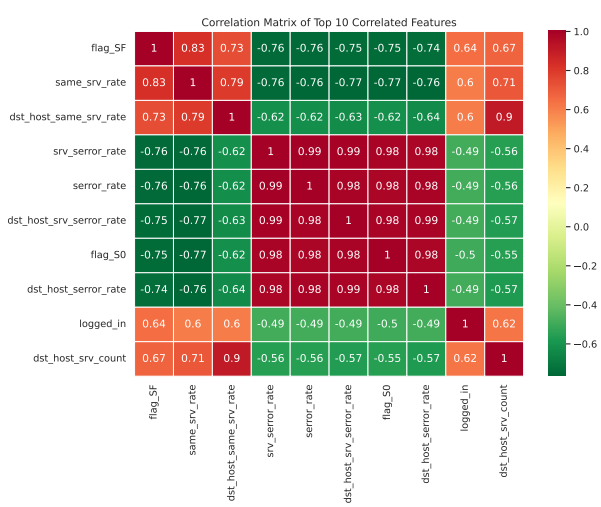


Fig. 5: Correlation Matrix

useful when dealing with correlated features, as it converts them into a new set of uncorrelated variables called principal components (PCs). This helps reduce computational complexity and improve the performance of machine learning models.

Given a dataset represented as a matrix $X \in \mathbb{R}^{n \times p}$, where n is the number of observations and p is the number of features. The PCA algorithm consists of the following steps:

1) Standardization: The first step is to normalize the dataset so that each feature contributes equally. The mean vector of the dataset is computed as:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i \tag{1}$$

The centered data matrix is obtained by subtracting the mean:

$$Y = HX \tag{2}$$

where H is the centering matrix.

2) Covariance Matrix Computation: The covariance matrix is calculated to measure the relationships between features:

$$S = \frac{1}{n-1} Y^T Y \tag{3}$$

3) Eigen Decomposition: The covariance matrix is decomposed into eigenvalues and eigenvectors:

$$S = A \Lambda A^T \tag{4}$$

where A is the matrix of eigenvectors and Λ is the diagonal matrix of eigenvalues. The eigenvectors represent the principal components, and the eigenvalues indicate the variance captured by each component.

4) Projection onto Principal Components: The original data is projected onto the new feature space defined by the principal components:

$$Z = YA \tag{5}$$

The matrix Z represents the transformed dataset in the reduced-dimensional space. The number of selected principal components is determined based on the explained variance ratio to retain the most informative features.

In this study, PCA is applied to the training dataset, and the learned transformation is subsequently applied to the testing dataset to ensure consistency. The effectiveness of PCA is further analyzed using visualization techniques such as explained variance plots, PCA scatter plots, and biplots.

E. E. Evaluation Metrics for Classification

To evaluate the performance of the classification models, several standard metrics are used, including accuracy, precision, recall, and F1-score [12].

Accuracy measures the overall correctness of the model and is defined as:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{6}$$

where TP is the number of true positives, TN is the number of true negatives, FP is the number of false positive and the FN is the number of false negative.

Precision represents the proportion of correctly predicted positive instances:

$$Precision = \frac{TP}{TP + FP} \tag{7}$$

Recall measures the ability of the model to detect positive instances:

$$Recall = \frac{TP}{TP + FN} \tag{8}$$

The F1-score combines both precision and recall into a single metric:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{9}$$

These metrics provide a comprehensive evaluation of the model performance, especially in the presence of imbalanced datasets.

F. F. Machine Learning Algorithms

In this study, multiple machine learning algorithms are implemented and compared using the PyCaret framework. PyCaret provides an efficient environment for training, tuning, and evaluating various classification models.

The models are evaluated under two scenarios: (i) using the original dataset without PCA, and (ii) using the PCA-transformed dataset. Several classifiers, including Extreme Gradient Boosting (XGBoost), Random Forest (RF), Extra Trees (ET), Decision Tree (DT), Logistic Regression (LR), and Support Vector Machine (SVM), are considered.

After model comparison, the best-performing model is selected and further optimized using hyperparameter tuning. Finally, Explainable Artificial Intelligence (XAI) techniques based on SHAP (Shapley Additive Explanations) are applied to interpret the model's predictions. SHAP provides both global and local explanations, allowing for a better understanding of feature importance and decision-making behavior.

1) Extreme Gradient Boosting (XGBoost): XGBoost is an advanced ensemble learning algorithm based on gradient boosting. It builds models sequentially by minimizing a differentiable loss function. The objective function is defined as:

$$\mathcal{L} = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k) \tag{10}$$

where l represents the loss function and $\Omega(f_k)$ is the regularization term controlling model complexity. XGBoost is known for its high performance and efficiency in handling large datasets [4].

2) Random Forest (RF): Random Forest is an ensemble learning method that constructs multiple decision trees and combines their predictions using majority voting. Each tree is trained on a random subset of features and samples, which improves generalization and reduces overfitting [6].

3) Extra Trees (ET): Extra Trees, or Extremely Randomized Trees, is similar to Random Forest but introduces more randomness by selecting split thresholds randomly instead of optimally. This reduces variance and improves computational efficiency [6].

4) Decision Tree (DT): Decision Tree is a supervised learning algorithm that splits the data based on feature values to create a tree-like structure. The decision is made by selecting features that maximize information gain. Decision Trees are simple and interpretable but prone to overfitting [6].

5) Logistic Regression (LR): Logistic Regression is a linear classification algorithm used to estimate the probability of a binary or multi-class outcome. The logistic function is defined as:

$$S(z) = \frac{1}{1 + e^{-z}} \tag{11}$$

where $z = w^T x + b$. The output ranges between 0 and 1, representing class probabilities. Logistic Regression is easy to implement but may struggle with non-linear data [6].

6) Support Vector Machine (SVM): Support Vector Machine is a powerful classification algorithm that finds an optimal hyperplane separating different classes. The objective is to maximize the margin between classes. Mathematically, the SVM model can be expressed using a linear decision function defined as:

$$f(x) = w \cdot x + b \tag{12}$$

where w represents the weight vector and b denotes the bias term. The function $f(x)$ determines the class of a given input x based on its position relative to the hyperplane.

The classification rule is defined as:

$$f(x) = \begin{cases} +1, & \text{if } f(x) \geq 0 \quad \text{(Normal class)} \\ -1, & \text{if } f(x) < 0 \quad \text{(Intrusion)} \end{cases} \tag{13}$$

In this formulation, data points located on or above the hyperplane are classified as normal, while those below the hyperplane are classified as intrusions [17]. SVM can also utilize different kernel functions to handle non-linear data. SVM performs well in high-dimensional spaces but may require careful parameter tuning [4].

7) K-Nearest Neighbour (KNN): K-Nearest Neighbour (KNN) is a supervised, non-parametric machine learning algorithm used for classification and pattern recognition. In KNN, a new data point is classified based on the majority class among its k nearest neighbours in the feature space. The similarity between data points is typically measured using Euclidean distance, defined as:

$$d(a, b) = \sqrt{\sum_{i=1}^{n} (a_i - b_i)^2} \tag{14}$$

where a and b are two data points and n represents the number of features. After computing the distances, the algorithm selects the k closest neighbours and assigns the class

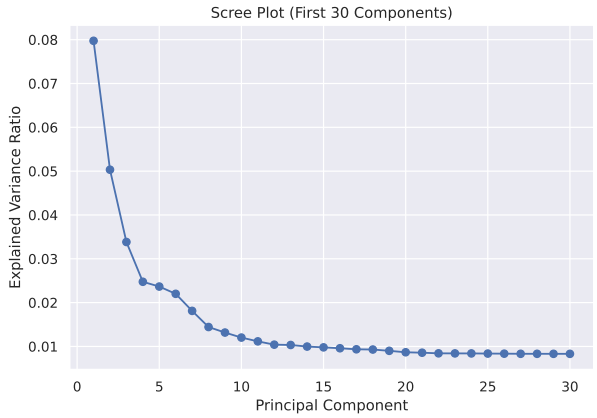


Fig. 6: Scree Plot

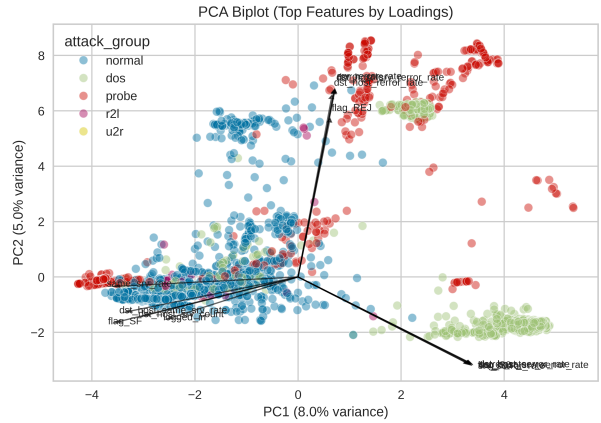


Fig. 8: biplot

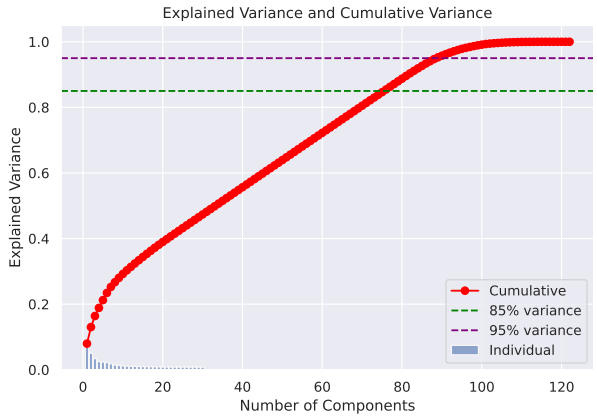


Fig. 7: Pareto Plot

label based on majority voting. KNN is simple and effective; however, its performance is sensitive to the choice of k and can be computationally expensive for large datasets [4], [17].

After evaluating all models, the best-performing classifier is selected and further optimized. Finally, Explainable Artificial Intelligence (XAI) techniques based on SHAP are applied to interpret the model predictions and analyze feature importance.

III. Simulation Results & Discussions

A. PCA Results

Principal Component Analysis (PCA) is applied to the NSL-KDD dataset to reduce the dimensionality of the feature space while preserving the most significant information. Due to the large number of features in the dataset, it is difficult to visualize and interpret all features simultaneously. Therefore, in this study, the PCA visualization is performed using the top 10 most correlated features to provide a clearer and more interpretable analysis. The complete analysis using all features is conducted and included in the Google colab notebook in github link.

The implementation of PCA is carried out using a standard machine learning library, which provides efficient computation of principal components, eigenvalues, and explained variance. The PCA transformation is fitted on the training dataset.

Fig. 6 presents the scree plot of the first 30 principal components, while the complete analysis of all components is provided in the Google Colab notebook. Fig. 6 presents the scree plot, which illustrates the explained variance ratio of each principal component. It can be observed that the first few principal components capture a significant portion of the total variance, with a sharp decline after the initial components. This indicates that most of the important information in the dataset is captured by the first few principal components. The first principal component (PC1) contributes the highest variance, followed by the second principal component (PC2). The elbow point in the scree plot indicates that only a few principal components are sufficient to represent the dataset effectively.

Fig. 7 shows the explained variance and cumulative variance plot. The cumulative variance curve demonstrates that approximately 85% of the total variance is captured within the first set of principal components, while around 95% variance is achieved with a larger number of components. This confirms that the dataset can be effectively represented in a reduced-dimensional space without significant loss of information.

The percentage of variance explained by each principal component is calculated using:

$$\eta_j = \frac{\lambda_j}{\sum_{i=1}^{p} \lambda_i} \times 100 \tag{15}$$

where λ_j represents the eigenvalue of the j-th principal component.

Fig. 8 illustrates the biplot of the first two principal components (PC1 and PC2), providing a combined visualization of both observations and feature loadings. The axes represent the first two principal components, while each data point corresponds to an observation colored according to its attack group. The vectors represent the contribution of the original features to the principal components.

It can be observed that the data points form distinguishable clusters for different attack groups, indicating that the first

two principal components capture meaningful patterns in the dataset. In particular, the DoS and probe classes show clearer separation along the PC axes, while normal traffic exhibits a wider spread.

The direction and length of the feature vectors indicate their contribution to the principal components. Features whose vectors are closely aligned with PC1 contribute more significantly to the first principal component, while those aligned with PC2 contribute more to the second component. For instance, features such as *serror_rate* and *dst_host_serror_rate* show strong alignment with one direction, indicating a higher influence on a specific principal component. Additionally, features pointing in similar directions are positively correlated, whereas features in opposite directions are negatively correlated.

Overall, PCA successfully reduces the dimensionality of the dataset while preserving the essential information. The analysis demonstrates that a reduced number of principal components can effectively represent the original dataset, thereby improving computational efficiency and enabling better visualization. Furthermore, the separation observed in PCA plots provides useful insights into class distribution and feature importance for intrusion detection.

B. Classification Results and Discussion

In this section, the performance of different machine learning classifiers for network intrusion detection is analyzed using the NSL-KDD dataset. The experiments are conducted before and after applying Principal Component Analysis (PCA) using the PyCaret framework, which enables efficient comparison, tuning, and evaluation of various machine learning models to evaluate the impact of dimensionality reduction on classification performance.

Table II presents the performance of different classification models before applying PCA. It can be observed that ensemble-based models such as XGBoost, Random Forest (RF), and Extra Trees (ET) achieve the highest performance, with accuracy values exceeding 99%. Among them, XGBoost achieves the best overall performance with an accuracy of 0.9992 and perfect AUC score of 1.0000. Similarly, RF and ET also demonstrate strong classification capability with very high precision, recall, and F1-scores. These results indicate that ensemble learning methods are highly effective in handling high-dimensional intrusion detection data.

Table III shows the performance of the same models after applying PCA. It is observed that the performance of some models slightly decreases due to dimensionality reduction, as PCA transforms the original feature space into a lower-dimensional representation. However, ET and RF continue to perform well, maintaining high accuracy values of 0.9943 and 0.9936, respectively. Interestingly, KNN shows a noticeable improvement after PCA, achieving an accuracy of 0.9919, indicating that distance-based models benefit from reduced feature dimensionality and noise removal. XGBoost, while still performing competitively, shows a slight reduction in accuracy compared to its performance before PCA.

These observations suggest that PCA helps in reducing redundancy and noise in the dataset, which improves the performance of certain models such as KNN, while slightly affecting highly complex models like XGBoost that rely on richer feature representations. Based on the results, ET, RF, KNN, and XGBoost are selected for further evaluation.

Fig. 9 presents the confusion matrices for Extra Tree, Random Forest, and KNN models. The confusion matrix provides a detailed comparison between predicted and actual class labels, where the horizontal axis represents the predicted class and the vertical axis represents the true class. illustrates the confusion matrices for Extra Tree, Random Forest, and KNN models. All three models perform well on majority classes such as Normal and DoS. For example, RF correctly classifies 9333 DoS instances and 5469 Normal instances, while KNN correctly predicts 8989 DoS and 5697 Normal samples. However, misclassifications are more noticeable in minority classes such as R2L and U2R. For instance, ET misclassifies a large portion of class 3 (R2L), with only 50 correct predictions compared to many misclassified as other classes. Among the models, RF shows slightly fewer misclassifications compared to ET and KNN, indicating better generalization.

Fig. 10 illustrates the learning curves for the selected models after applying PCA, including Extra Trees (ET), Random Forest (RF), and K-Nearest Neighbour (KNN). The plots show the relationship between the number of training instances and model performance, where the blue line represents the training score and the green line represents the cross-validation score. It can be observed that both ET and RF achieve near-perfect training accuracy, close to 1.0, indicating strong fitting to the training data. Their cross-validation scores gradually improve as the training size increases and stabilize around 0.993–0.994, showing consistent generalization performance. KNN, on the other hand, exhibits slightly lower training accuracy compared to ET and RF, but its cross-validation score steadily increases and reaches comparable performance as more data is introduced. The gap between the training and validation curves is relatively small for all models, indicating that overfitting is minimal and the models generalize well to unseen data. Additionally, the smooth convergence of the curves suggests that increasing the training data contributes to improved performance and model stability. Overall, ET and RF demonstrate more stable and consistent learning behavior, while KNN shows gradual improvement with increased data, reflecting its sensitivity to data distribution.

Fig. 11 presents the ROC curves and illustrates the trade-off between the True Positive Rate (TPR) and False Positive Rate (FPR), while the Area Under the Curve (AUC) provides an overall measure of classification performance. It can be observed that ET and RF achieve strong performance across most classes, with AUC values generally above 0.90. For instance, both models achieve AUC values of approximately 0.92 for class 0 and up to 0.96 for class 2, indicating excellent discrimination capability. In contrast, KNN shows comparatively lower performance, with AUC values around

TABLE II: Model Performance before PCA

Model	Acc	AUC	Rec	Prec	F1	TT
XGB	0.9992	1.0000	0.9992	0.9992	0.9992	7.4050
RF	0.9989	1.0000	0.9989	0.9989	0.9989	8.1960
ET	0.9986	0.9999	0.9986	0.9985	0.9985	7.5790
DT	0.9980	0.9984	0.9980	0.9980	0.9980	1.3550
GBC	0.9971	0.0000	0.9971	0.9973	0.9972	79.1450
KNN	0.9926	0.9981	0.9926	0.9926	0.9925	13.7490
LGBM	0.9667	0.9716	0.9667	0.9773	0.9707	12.9140
RIDGE	0.9522	0.0000	0.9522	0.9446	0.9482	0.6900
LDA	0.9491	0.0000	0.9491	0.9532	0.9507	1.0390
LR	0.8399	0.0000	0.8399	0.8166	0.8044	44.3230
ADA	0.8231	0.0000	0.8231	0.8688	0.8155	4.6140
QDA	0.6987	0.0000	0.6987	0.9145	0.7506	1.0020
DUMMY	0.5346	0.5000	0.5346	0.2858	0.3725	0.6870
NB	0.3870	0.8754	0.3870	0.5839	0.2625	0.7860
SVM	0.2255	0.0000	0.2255	0.4396	0.2923	2.1720

TABLE III: Model Performance after PCA

Model	Acc	AUC	Rec	Prec	F1	TT
ET	0.9943	0.9992	0.9943	0.9942	0.9942	3.7330
RF	0.9936	0.9994	0.9936	0.9934	0.9935	11.9970
KNN	0.9919	0.9981	0.9919	0.9918	0.9918	1.6250
XGB	0.9915	0.9996	0.9915	0.9913	0.9914	3.7320
DT	0.9901	0.9920	0.9901	0.9900	0.9900	1.4900
LGBM	0.9853	0.9946	0.9853	0.9864	0.9858	7.1640
GBC	0.9821	0.0000	0.9821	0.9820	0.9819	56.5050
QDA	0.8912	0.0000	0.8912	0.8864	0.8876	1.0860
LR	0.8909	0.0000	0.8909	0.8764	0.8826	2.2320
NB	0.8906	0.9573	0.8906	0.8850	0.8876	1.1550
RIDGE	0.8768	0.0000	0.8768	0.8516	0.8538	1.0970
SVM	0.8740	0.0000	0.8740	0.8471	0.8543	1.7440
LDA	0.8268	0.0000	0.8268	0.8654	0.8373	1.0640
DUMMY	0.5346	0.5000	0.5346	0.2858	0.3725	1.0170
ADA	0.4659	0.0000	0.4659	0.4624	0.4444	3.8840

0.85 for class 0 and dropping to approximately 0.53–0.54 for minority classes such as class 3 and class 4. Overall, the ROC analysis confirms that ET and RF outperform KNN in terms of classification robustness and consistency, particularly in handling imbalanced classes, making them more suitable for intrusion detection tasks on the NSL-KDD dataset.

Overall, the results demonstrate that ensemble-based models such as Random Forest and Extra Trees provide robust and consistent performance for intrusion detection. PCA proves beneficial in reducing dimensionality and improving the performance of simpler models such as KNN, while maintaining competitive performance for complex models. However, the class imbalance in the dataset remains a significant challenge, particularly for detecting rare attack types such as R2L and U2R.

C. EXPLAINABLE AI WITH SHAPLEY VALUES

Model interpretability is essential to understand how individual features contribute to the prediction of machine learning models. In this study, SHAP (Shapley Additive Explanations) is used to explain the behavior of the selected model after applying PCA. SHAP is based on game theory and assigns each feature an importance value, representing its contribution to the final prediction.

Fig. 12 shows the SHAP summary plot for the Random Forest model using PCA-transformed features. In this case, each principal component (PC) acts as a feature. The plot illustrates the mean absolute SHAP values, which indicate the overall importance of each component. It can be observed that *pca0* has the highest impact on the model output, followed by *pca2* and *pca1*. This indicates that the first principal component carries the most significant information for classification, which is consistent with the variance explained by PCA.

The color distribution in the plot represents the contribution of each class, showing how different components influence the prediction across multiple classes. Higher SHAP values correspond to a stronger influence on the model output, confirming that the model relies heavily on the first few principal components rather than all original features.

Fig. 13 presents the SHAP force plot for a single observation. This plot explains how each principal component contributes to the prediction of that specific instance. The base value represents the average model output, while the final prediction is obtained by adding the contributions of each feature. In this example, *pca2* contributes positively to increasing the prediction score, whereas *pca1* and *pca0* contribute negatively, pushing the prediction towards a lower value.

The red regions in the force plot indicate features that push the prediction higher, while the blue regions indicate features that push it lower. The balance of these contributions determines the final classification of the instance. This analysis demonstrates how PCA components influence model decisions and provides transparency into the model's prediction process.

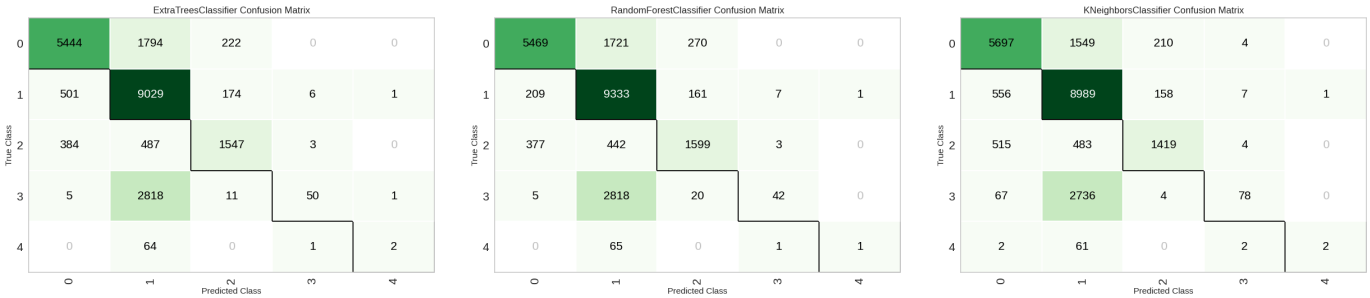
CONCLUSION

In this study, a comprehensive machine learning framework for network intrusion detection was developed using the NSL-KDD dataset. The dataset was preprocessed through categorical encoding and feature standardization, followed by dimensionality reduction using Principal Component Analysis (PCA). The effectiveness of PCA was evaluated by comparing the performance of multiple classification models before and after dimensionality reduction.

The results show that ensemble models such as Random Forest (RF) and Extra Trees (ET) achieve consistently high performance, while XGBoost performs best before PCA. After applying PCA, KNN shows noticeable improvement, indicating that dimensionality reduction helps reduce noise and improves distance-based learning.

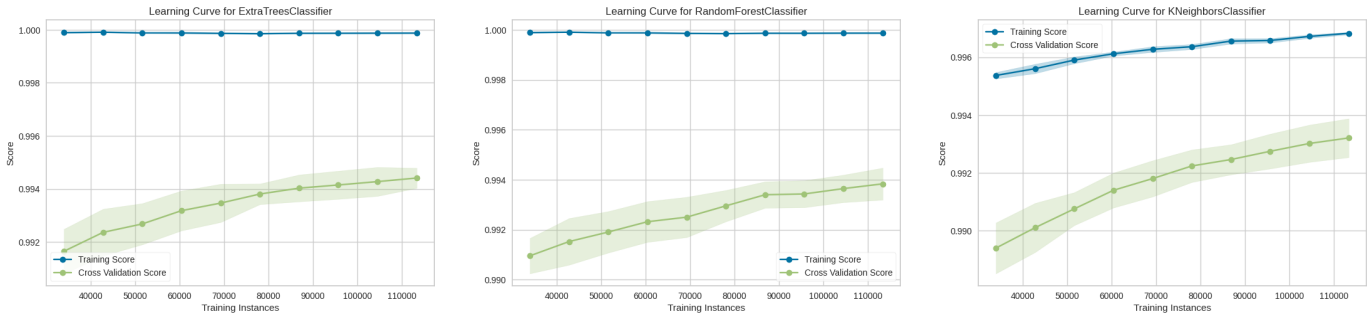
The proposed approach demonstrates that PCA effectively reduces feature dimensionality while maintaining strong classification performance. However, the models show lower performance on minority classes such as R2L and U2R due to class imbalance. SHAP analysis further confirms that the first principal component contributes most to the prediction, improving model interpretability.

In future work, class imbalance can be addressed using techniques such as SMOTE or class-weighted learning. Advanced methods such as autoencoders, deep learning models, and ensemble techniques can be explored to further enhance performance. Additionally, evaluating the model on more



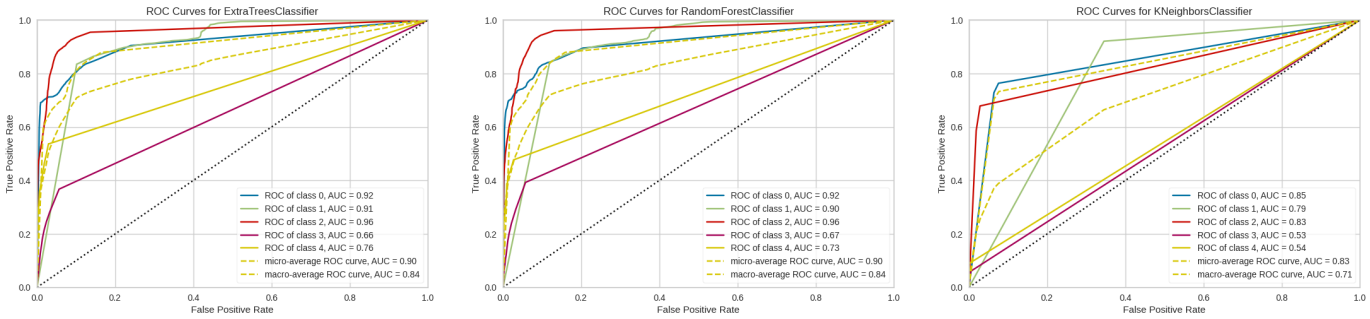
(a) (b) (c)

Fig. 9: Confusion Matrix (a) Extra Tree , (b) Random forest, and (c) KNN



(a) (b) (c)

Fig. 10: Learning curve (a) Extra Tree , (b) Random forest, and (c) KNN



(a) (b) (c)

Fig. 11: Class report (a) Extra Tree , (b) Random forest, and (c) KNN

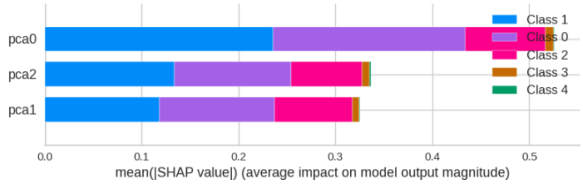


Fig. 12: SHAP value

Fig. 13: Force plot for a single observation

recent datasets and real-time environments can improve its applicability to practical intrusion detection systems.

References

[1] S. Kumar and E. H. Spafford, "A pattern matching model for misuse intrusion detection," in *Proc. National Computer Security Conference*, 1994.

[2] D. E. Denning, "An intrusion detection model," *IEEE Transactions on Software Engineering*, vol. 13, no. 2, pp. 222–232, Feb. 1987.

[3] H. J. Liao, C. H. R. Lin, Y. C. Lin, and K. Y. Tung, "Intrusion detection system: A comprehensive review," *Journal of Network and Computer Applications*, vol. 36, no. 1, pp. 16–24, 2013.

[4] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symposium on Security and Privacy*, 2010, pp. 305–316.

[5] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symposium on Computational Intelligence for Security and Defense Applications*, 2009, pp. 1–6.

[6] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *Journal of Machine Learning Research*, vol. 3, pp. 1157–1182, 2003.

[7] I. T. Jolliffe, *Principal Component Analysis*. New York, NY, USA: Springer, 2002.

[8] S. M. Lundberg and S. I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[9] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[10] A. Goldstein, A. Kapelner, J. Bleich, and E. Pitkin, "Peeking inside the black box: Visualizing statistical learning with plots of individual conditional expectation," *Journal of Computational and Graphical Statistics*, vol. 24, no. 1, pp. 44–65, 2015.

[11] "NSL-KDD dataset", https://www.kaggle.com/datasets/hassan06/nslkdd/data.

[12] R. O. Ogundokun, M. Odusami, D. S. Sisodia, J. B. Awotunde, and D. P. Tiwari, "A Novel PCA-Logistic Regression for Intrusion Detection System," in *Data Science and Applications*, Springer, 2023.

[13] Z. H. Salim and S. O. Hasoon, "Improving Machine Learning-Based Intrusion Detection Systems: A Comparative Study on NSL-KDD Dataset," in *Innovations of Intelligent Informatics, Networking, and Cybersecurity*, Communications in Computer and Information Science, vol. 2329, Springer, Cham, 2025.

[14] R. Abdulhammed, H. Musafer, A. Alessa, M. Faezipour, and A. Abuzneid, "Features Dimensionality Reduction Approaches for Machine Learning Based Network Intrusion Detection," *Electronics*, vol. 8, no. 3, p. 322, 2019.

[15] O. Chentoufi, M. Choukhairi, K. Chougdali, and I. Alloug, "Exploring the Impact of PCA Variants on Intrusion Detection System Performance," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 15, no. 5, 2024.

[16] Yuliana, D. H. Supriyadi, M. R. Fahlevi, and M. R. Arisagas, "Analysis of NSL-KDD for the Implementation of Machine Learning in Network Intrusion Detection System," *Journal of Informatics, Information System, Software Engineering and Applications (INISTA)*, vol. 1, no. 1, pp. 001–010, 2023.

[17] A. D. Vibhute, C. H. Patil, A. V. Mane, and K. V. Kale, 'Towards Detection of Network Anomalies using Machine Learning Algorithms on the NSL-KDD Benchmark Datasets," *Procedia Computer Science*, vol. 233, pp. 960–969, 2024.